



SMART PLANT MONITORING

Self-Provisioning Setup

INTERNAL · MAIL-A-BOARD

Version 1.0 · June 2026 · one image, every board its own device

GrowthPulse Self-Provisioning Setup

How to turn on "mail a customer a board." Every board runs ONE identical firmware (`GP_Node`, no per-board secrets). On first boot a board registers itself: it sends its pairing code + a shared firmware token to a backend endpoint, gets its own Losant device + access key, and saves them to flash. Flash 1 board or 1,000, each shows up in the app as its own plant; the customer just claims it with the code on the screen.

This replaces the old model where the Losant device id was hard-coded into the firmware (so every board was the same device). Your existing demo unit keeps its identity automatically (its code is already in the registry).

You set this up once. After that, new boards need zero manual cloud work.

What's involved

```
Board (first boot) --POST {code, token}--> provision-device (Netlify)
                                     |-- creates a Losant device (all attributes defined)
                                     |-- inserts code -> device id into the registry (Supabase)
                                     |-- mints an access key scoped to that device
                                     <-- { deviceId, accessKey, accessSecret }
Board saves creds to flash (NVS) and connects to Losant as normal.
Customer claims the board in the app by the code on its screen.
```

One-time setup (you, ~15 minutes)

1. Create a Losant API token that can make devices + keys

Losant Console → your Application → **Security** → **Application API Tokens** → **Add Application API Token**. Give it the scopes `devices.*` and `applicationKeys.*` (or simply `all.Application`). Copy the token (shown once).

2. Get your Supabase service-role key

Supabase → your project → **Project Settings** → **API** → **Project API keys** → `service_role` (the secret one, not anon). Copy it. The function uses it to read/insert the registry (which is RLS-protected against the public key).

3. Set the Netlify environment variables

Netlify → your site → **Site configuration** → **Environment variables**. Add:

Variable	Value
PROVISION_TOKEN	a long random secret you choose (the firmware carries the same value)
LOSANT_PROVISION_API_TOKEN	the Losant API token from step 1
SUPABASE_SERVICE_ROLE_KEY	the service-role key from step 2

Already set (confirm they exist): `LOSANT_APP_ID`, `VITE_SUPABASE_URL`.

4. Put the shared token in the firmware

Open `GP_Node.ino` and set:

```
#define PROVISION_TOKEN "REPLACE-WITH-SHARED-FIRMWARE-TOKEN" // must equal Netlify PROVISION_TOKEN
```

to the same string you used for `PROVISION_TOKEN` in Netlify. This is the **ONLY** value you edit, and it's identical for every board you ever flash.

5. Push so the function deploys

```
cd "Senior Design 2/growthpulse-app"
git add . && git commit -m "Self-provisioning: provision-device function + GP_Node firmware" && git push
```

(Do it on your `lorawan-bringup` branch or a fresh branch; merge to `main` when you're happy. Netlify builds on push.)

Flash a board

- Open `GP_Node.ino` in Arduino, board = Heltec WiFi LoRa 32 V3, flash it.
- Do the normal Wi-Fi setup (join its hotspot, pick your Wi-Fi).
- Watch the Serial Monitor. First boot prints **"First-time setup / registering device"** then **"Provisioned. Losant device id: ..."**. Every later boot prints **"Loaded saved Losant identity: ..."**.
- In Losant you'll see a new device named **GrowthPulse <CODE>** appear, with all attributes defined (including `wifiRssi`, so you no longer add that by hand).
- In the app, **Connect a device** and enter the code on the board's screen. It shows up as its own plant.

Flash a second board: same image, nothing to change. It self-registers as a separate device.

How your existing board stays the same

The demo unit's code (`4A7AC`) is already in the registry pointing at its existing Losant device. When the provisioning endpoint sees a code that's already registered, it reuses that device and just mints a fresh key. So reflashing your current board with `GP_Node` keeps all its history; it does not create a duplicate.

Verify it worked

- Netlify → **Functions** → **provision-device** → **logs**: a **200** with the new device id on first boot.
- Losant: the new **GrowthPulse <CODE>** device receiving state.
- App: the claimed board shows live readings.

Errors: **401 Bad provisioning token** → the firmware **PROVISION_TOKEN** and the Netlify env var don't match. **502 Losant device create failed** → the Losant API token is missing the **devices.* / applicationKeys.*** scope. **502 Registry insert failed** → the Supabase service-role key is wrong or missing.

Security notes (honest)

- The shared firmware token is the same on every board. If someone extracts it from a board, they could call the provisioning endpoint. For a shipping product you'd harden this (per-batch tokens, a hardware secure element, or signed device certificates). For now it only allows creating a device + a key scoped to that one device, so the blast radius is small; add rate limiting if you expose it widely.
- The board uses TLS without certificate pinning (**setInsecure**) to keep the bring-up simple. Pin the certificate (or use a known root CA) before production so the provisioning response can't be intercepted by an active man-in-the-middle.
- This is the productized version of roadmap item #1 (per-unit provisioning) and item #3 (zero-typing auto-bind) from the Engineering Manual.

GrowthPulse Engineering, internal. Self-provisioning setup.